

# KATHDB-FAO: Synthesized Query Plans in a Multimodal DBMS

Guorui Xiao  
University of Washington  
grxiao@cs.washington.edu

Artur Borycki  
Teradata  
Artur.Borycki@teradata.com

Douglas Brown  
Teradata  
Doug.Brown@teradata.com

Magdalena Balazinska  
University of Washington  
magda@cs.washington.edu

## ABSTRACT

We design, implement, and evaluate **KATHDB-FAO**, a new query evaluation subsystem for our KathDB multimodal DBMS. **KATHDB-FAO** takes as input a query in natural language (NL) and converts it into a query execution plan where each operator is a function whose body is synthesized during query evaluation, which allows powerful query-specific optimizations. To generate accurate and efficient plans from NL, **KATHDB-FAO** first extracts fine-grained atomic actions for correctness, then establishes contracts on the inputs and outputs of those actions and groups them for efficiency, and finally synthesizes the function for each group on the fly. On SemBench, **KATHDB-FAO** cuts execution cost by 58.8% on average across scenarios compared with the next best system, at comparable or better quality.

### PVLDB Reference Format:

Guorui Xiao, Douglas Brown, Artur Borycki, and Magdalena Balazinska. **KATHDB-FAO: Synthesized Query Plans in a Multimodal DBMS**. PVLDB, 14(1): XXX-XXX, 2020. doi:XX.XX/XXX.XX

### PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/xertxiao/KathDB>.

## 1 INTRODUCTION

Advances in artificial intelligence (AI), specifically computer vision [10, 26] and natural language processing [8, 35] have enabled the extraction and querying of the semantic content of unstructured data types, leading to the re-emergence of new types of DBMSs specialized in processing such data including text [5, 29], images [19], and videos [12, 13, 42]. One sign of the increased practicality of modern image and text processing capabilities is their adoption in commercial, traditionally only relational DBMSs [9, 30]. Nevertheless, important challenges remain. In particular, companies and applications that want to process unstructured data typically have access to more than one type of data [9]. For example, in a medical setting, patient records include relational data (e.g., annual cholesterol measurements), medical images (e.g., x-rays or

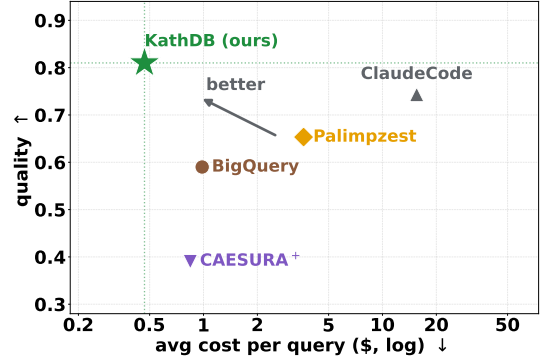


Figure 1: KATHDB-FAO achieves the best overall quality at the lowest or matching overall cost, including plan generation cost, across all systems, despite taking NL input and generating query plans on the fly, whereas BigQuery and Palimpzest require that the user write SQL or procedural plan code.

ultrasounds), and doctor notes in text format. Recent multimodal DBMSs [11, 17, 21, 23, 29] focus on managing such collections of structured and unstructured data in one system. These new types of DBMSs are in their infancy, however, and there is still no agreement on the best query language, data model, nor query execution engine for these systems, with various experimental prototypes being developed. While some systems expose SQL [9, 30] or programmatic [17, 23, 29] interfaces, other systems support Natural Language (NL) queries [21, 33], which makes them easier to adopt by a broader class of users. General-purpose agents, such as coding agents [3] and deep research systems [22, 24], can also answer NL questions over data by writing and running code. They are not query engines, however: they neither optimize the plan they execute nor reuse work across queries, and we show in §6 that this makes them much more expensive than our system at a comparable level of quality.

In this paper, we design, implement, and evaluate **KATHDB-FAO**, a new query evaluation subsystem part of our **KATHDB** multimodal DBMS. In **KATHDB**, users submit queries in natural language (NL), which facilitates the combined querying of data across modalities and relieves the user from the requirement to master a structured query language. **KATHDB-FAO** converts an NL query into a query execution plan that it optimizes and executes over data that includes relations, images, and text. Building such an engine is difficult for three reasons, which we illustrate with a running example: consider a company that requires employees to wear uniforms at work (e.g.,

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing [info@vldb.org](mailto:info@vldb.org). Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment. Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097. doi:XX.XX/XXX.XX

some coffee shops). The company wants a supplier that is affordable and offers enough suitable options (also shown in Figure 4):

**Example 1.** *We are picking a supplier for employee uniforms. Which brands sell everything under \$50 and offer at least five formal, non-patterned top-wear items? For each such brand, also count the main sentiments customers express about its garments.*

Answering the above query requires (1) a price check per brand, keeping only brands whose most expensive item is under \$50, and a category filter keeping their top-wear; (2) a scan of the remaining items, using images and descriptions to decide whether each one is formal and non-patterned; (3) a per-brand count that keeps brands with at least five qualifying items; and (4) for each such brand, a scan of its reviews to extract the sentiments and count their frequencies.

**Challenge 1: In multimodal DBMSs, there is no canonical translation from an NL query to a query plan.** For example, step (2) has no predefined operator in a relational engine, and its semantics cannot be captured by classical relational algebra. Existing systems address this limitation in two ways, and neither is sufficient. Operator-based systems such as Palimpzest [17] and LOTUS [23] expose a fixed set of semantic operators that hard-code LLM inference over data records and require that the user decompose their query into those operators by hand. This places the burden on the user, who must know both the operator interface and the data. It also does not guarantee a good plan: even expert-authored plans, such as those written for SemBench [15], can be far more expensive than necessary (§6). Some early systems translate NL queries into plans automatically: CAESURA [33] uses an LLM to map an NL query onto a fixed set of operators, and VOCAL-UDF [43] synthesizes a function for a missing operation on demand. Neither optimizes the resulting plan to reduce its execution cost.

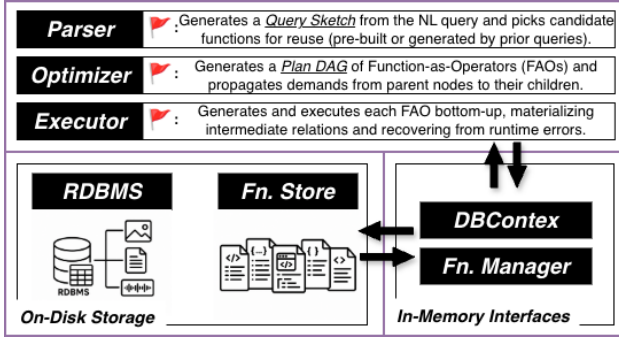
**Challenge 2: Query optimization in a multimodal DBMS is challenging.** Semantic reasoning over data requires model inference, which is slow and expensive [39, 43]; in multimodal DBMSs it is typically the most expensive operation, and the best plans are those that minimize it [17, 23]. A naive plan for Example 1 runs the vision model on every top-wear item. A better plan checks prices first, which needs no model call and removes most brands. The best plan goes further: the query needs only five qualifying items per brand, so it classifies a brand’s items only until five qualify. Such multimodal plans are challenging for a traditional query optimizer. First, the optimizer has no cost estimate for the semantic step, whose selectivity depends on model behavior [27, 39], which limits query optimization. Second, the best plan interleaves aggregation and model inference, stopping the latter for specific brands once those brands reach the count of five, which is a difficult optimization in a traditional engine that composes query plans from predefined operators. Existing multimodal systems [9, 17, 23, 29, 33] cannot achieve this because they run semantic steps as bulk operators over the whole collection. Third, this is one rewrite among many a query may need, and each system supports only the fixed set its designers built in, whereas KATHDB-FAO synthesizes the code of each operator and can realize query-specific rewrites that no fixed rule set contains.

**Challenge 3: Plan generation from NL is expensive.** Modern LLMs can translate an NL query into a query plan [33] and generate code from an NL query, but each such translation costs tokens

and latency. This cost is paid per query, even though queries in a workload can overlap heavily. For example, a user who lowers the threshold in Example 1 from five items to three changes one constant, yet a system that regenerates the plan from scratch per query pays the full planning cost again, which is wasteful and dominates overall query cost on small-scale data (§6).

**Our approach.** KATHDB-FAO takes as input a multimodal, natural language query, generates and optimizes a logical plan, then generates and executes a physical plan. In this paper, we focus on the logical query plan optimizer. Each plan node is a function that KATHDB-FAO synthesizes on the fly rather than an operator drawn from a fixed algebra, so a plan can express operations that have no relational counterpart and also operations that do not yet exist in the engine. Because the optimizer reasons about the code inside each query plan node, rather than treating it as an opaque user-defined function (UDF), it can fuse semantic steps with relational operations as needed, leading to more efficient plans that can exploit various early termination opportunities. Finally, KATHDB-FAO judiciously caches the functions it synthesizes and reuses them in later queries, such that workload-specific optimizations only need to be identified once, and then can be reused across queries. As shown in Figure 1, KATHDB-FAO achieves comparable or better quality at lower overall cost than all baselines, including commercial systems and those requiring human experts to draft the plan procedurally, even after accounting for the plan-generation cost (in USD) as LLM is involved. We revisit this figure in §6.1. More specifically, this paper makes the following contributions:

- **Function-as-Operator (FAO), a plan representation built on code generation (§2).** In KATHDB-FAO, each query plan operator is a function whose body is synthesized during query evaluation, enabling query-specific optimizations. We initially proposed this idea in a CIDR vision paper [41]. We develop the idea in full in this paper. We describe the system architecture and Function-as-Operator abstraction in §2.
- **An optimizer that groups atomic actions into fused operators (§3, §4).** We show empirically that synthesizing an entire query as a single function does not lead to the most efficient plans because it misses optimization opportunities that arise when the optimizer can see and reason about intermediate results. Instead, KATHDB-FAO introduces a new multimodal optimization approach that consists of parsing queries into semantically atomic actions, then optimizing plans by deriving contracts between adjacent actions and selecting actions to group into functions, before synthesizing code. KATHDB-FAO defines a search-space and cost-model based on optimizing expensive inference operations. We also present and evaluate different approaches to plan enumeration and cost comparison.
- **Code generation with function reuse (§5).** Code generation is both expensive and non-deterministic: the same plan may not always yield the same efficient implementation. Caching all generated functions, however, is also expensive as the LLM has to analyze an increasingly large cache during query evaluation and some functions may not capture the most efficient implementations. We develop



**Figure 2: Architecture of KATHDB.** DBContext registers and ingests data. Parser takes an NL query and produces a query sketch. Optimizer converts the sketch into a logical plan. Executor processes the DAG bottom-up, interleaving code generation with query evaluation, optionally using candidate functions registered in the Function Manager (Fn. Mgr.).

an approach that optimizes how functions are cached and reused.

- **Empirical evaluation on SemBench workloads (§6).** We evaluate **KATHDB-FAO** on the SemBench benchmark [15] across four scenarios. **KATHDB-FAO** achieves the best or tied-best quality on three of the four scenarios and is within 0.02 of the best on the fourth, while reducing execution cost by 58.8% on average (up to 85%) compared with the next best system.

## 2 SYSTEM ARCHITECTURE AND FUNCTION-AS-OPERATOR (FAO)

Figure 2 shows the architecture of **KATHDB** and its end-to-end workflow. A user first registers and ingests data: The **DBContext** component loads relational data into an underlying RDBMS.<sup>1</sup> It leaves multimodal data in files, registering only their file paths. Once data is registered, given an NL query, **KATHDB** processes it by invoking **KATHDB-FAO**, which proceeds in three stages:

**Parser** (§3): ❶ converts the query into a sequence of *natural-language* actions that serve as a high-level execution plan, which we call a *query sketch*. For each action, **KATHDB** may already have some relevant functions that could be used in the implementation of that action. The Parser ❷ identifies any such functions via the Function Manager (§5).

**Optimizer** (§4): ❸ transforms the query sketch into a logical plan: a directed acyclic graph (DAG) of FAO nodes. Each FAO is a function that takes one or more relations as input and outputs one or more relations. Although **KATHDB** is a multimodal DBMS, its plan representation remains grounded in the relational model.

**Definition 1 (Function-as-Operator (FAO)).**

$$FAO = \langle ID, desc, in, out, impl \rangle.$$

As shown in Figure 3, an FAO has five components: a unique *identifier* (*ID*) for a *group*, which is a set of one or more query-sketch actions assigned to this FAO, a *description* of the logic the FAO must realize, the input and output *schemas*, and an *implementation*. The

Parser emits the schema information for individual actions in the query sketch. The Optimizer builds this plan by first mapping each action of the query sketch into its own FAO; we call the result the *initial* or *atomic* plan. Starting with the atomic plan, the Optimizer ❷ establishes *contracts* between FAO nodes by propagating demands from each parent to its children: a parent specifies properties its inputs must satisfy and delegates that obligation to the producing child. For example, in Example 1 the per-brand count needs only to know whether each item is formal and non-patterned, so it demands that the two classifiers emit their labels from a fixed vocabulary ({Formal, Non-Formal} and {Patterned, Non-Patterned}) instead of describing any style (which could be business casual, smart, or striped). Thus, the count node can expect exact labels and count the qualifying items with a simple string test (e.g., `formal == 'Formal'` and `pattern == 'Non-Patterned'`), with no model call. The Optimizer ❸ then profiles the initial plan: it draws a small sample from the base relations, generates code for the atomic FAOs, and runs the plan to materialize all intermediate results, so later decisions are grounded in observed data rather than declared schemas alone. The Optimizer ❹ then applies logical rewrites to reduce the number of model calls (e.g., to a vision model [10] or an LLM [35]) over the data. The logical rewrites group atomic FAOs into more complex ones. In Example 1, the atomic plan selects the affordable brands, applies the vision model to every top-wear item of those brands, and then counts per brand and keeps brands with at least five qualifying items. Grouping the two classification FAOs, the per-brand count, and the `count ≥ 5` filter into one FAO lets the synthesized program interleave them and stop classifying a brand’s items at the fifth qualifying one. While modern DBMSs merge certain operations for efficiency (e.g., limit with sort to compute a top-*k* without a full sort), the set of merges is fixed; our approach enables the synthesis of query-specific optimizations. After these rewrites, remaining atomic FAOs already carry an implementation from profiling, while newly formed, grouped FAOs do not; the Optimizer defers their synthesis to the Executor.

**Executor** (§5): executes the plan, processing FAOs bottom-up. For each FAO, the Executor ❶ reuses its profiled implementation when one exists, and otherwise synthesizes a new one, potentially reusing relevant functions identified by the Parser. It then ❷ executes the FAO, materializing its output relation(s) at full scale. Because generated code can fail (e.g., a `KeyError` on a non-existent column), the Executor detects errors via runtime exceptions raised during execution and recovers by regenerating the function.

Answering an NL query with a DAG of FAOs raises several important questions: At what granularity should the Parser produce actions (§3)? How should the Optimizer group actions into FAOs (§4)? How should the system best cache and reuse past function implementations (§5)? We answer those questions in the following sections.

## 3 PARSER: DECOMPOSING NL QUERIES

The **KATHDB-FAO** Parser converts an NL query into a *query sketch*: a sequence of natural-language *actions* that capture the transformations needed to answer the query while deferring implementation

<sup>1</sup>In our prototype, we use DuckDB [25].

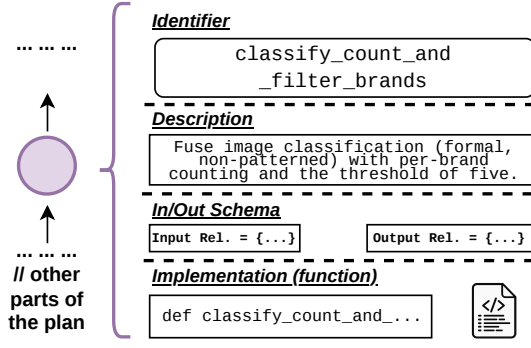


Figure 3: Example of a Function-as-Operator (FAO).

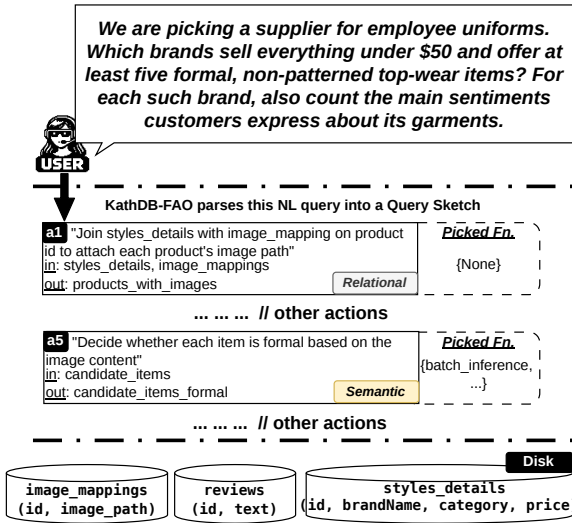


Figure 4: Parser decomposes an NL query into a query sketch.

decisions to later stages (Figure 4), similar to chain-of-thought reasoning [38] but with the additional fields defined below.

**Definition 2 (Action).** An *action* is the basic unit produced by the Parser:

$$a = \langle \text{name}, \tau, \text{desc}, \text{in}, \text{out}, \text{fns} \rangle,$$

where *name* identifies the action within a query, *desc* describes the transformation in natural language, *in* and *out* are its input and output relations, and *fns* is the set of candidate functions (pre-built or generated by earlier queries) the Parser flags as relevant for later code generation (§5). The category  $\tau$  is RELATIONAL for standard relational operations (e.g., join) with well-defined semantics and no model inference, or SEMANTIC when the action requires model-based interpretation of multimodal or unstructured content. In Figure 4, a5 is SEMANTIC, and the Parser flags *batch\_inference* as a relevant function.

**Q1: At what granularity should a query be decomposed?** Granularity decides the scope of a function. A larger scope makes the Code Generator more likely to make mistakes and yields less reusable code, but it enables optimizations that are only possible when the code is generated together. Choosing the best scope is

hard: unlike SQL, which translates into a fixed operator set, an NL query has no canonical decomposition. We consider three options: *Single-action granularity*. One function answers the entire query, leaving the most room for optimization since all logic resides in one FAO. However, it is harder to generate efficient code without intermediate data: the Code Generator must assume the active domains of intermediate relations. Assumptions that are too conservative easily lead to suboptimal implementations (e.g., a single function that must be ready to handle a larger variety of input values decides to use pairwise LLM comparison); assumptions that are too aggressive can lead to incorrect query outputs (§6.2) and occasionally to function regeneration after a runtime error (§6.3). Functions that encapsulate more logic may also be harder to reuse.

*LLM-decided granularity*. With this approach, the Parser lets the LLM pick the granularity it sees fit. As we show experimentally, the LLM still tends to choose actions that are too coarse, hitting the same two problems as above.

*Atomic granularity*. We develop an alternate approach in this paper: We have the Parser decompose the query into the finest-grained actions that still omit implementation details. We call those actions *semantically atomic*, and defer the scope decision (how many actions should form one FAO) to the Optimizer (§4).

**Definition 3 (Semantically atomic action).** An action *a* is *semantically atomic* if there do not exist two other actions, *a*<sub>1</sub> and *a*<sub>2</sub>, where (i) the output of *a*<sub>1</sub> is the input to *a*<sub>2</sub>, (ii) *a* is semantically equivalent to applying *a*<sub>1</sub> followed by *a*<sub>2</sub>, and (iii) neither action’s description exposes physical implementation details (e.g., prompt construction, model selection, remote service calls).

Decomposing a query into atomic actions enables two kinds of optimization: (1) additional reasoning on closed-vocabulary intermediate relations, which the Optimizer turns into contracts (§4, Q3), and (2) seeing samples of open-vocabulary intermediate relations, which we illustrate here. Consider the second part of Example 1: for each selected brand, count the main sentiments in its reviews. A single action answers this with one function written before anything has run. The Code Generator has no extracted sentiments in hand and cannot know how they would be spelled, so the implementation that is correct under every unknown keeps the model in the loop: it asks the LLM, once per pair of reviews, whether they express the same sentiment,  $O(n^2)$  calls. The atomic Parser instead splits this part of the query sketch into two actions: extract the main sentiment of each review, then merge equivalent sentiments and count them per brand. Materializing the first action’s output (the full relation, or the profiled sample) lets the Code Generator see the extracted sentiments, which are often semantically the same yet differ in text (e.g., comfortable, cozy), and map them to one canonical value with a dictionary before counting, falling back to the LLM only for values not in the dictionary. The single action cannot do this as the entire logic needs to be generated at once. Unlike the formal and non-patterned labels, whose vocabulary the count node fixes by contract before any code runs (§2), sentiment is an open domain: no consumer can name its values in advance. We evaluate parsing granularity in §6.2 and find that atomic actions yield the highest quality on hard queries with lower execution cost, but at a higher planning cost, since the system now generates code

Table 1: Notation used in §4.

| Symbol                | Meaning                                                             |
|-----------------------|---------------------------------------------------------------------|
| $a, A, N$             | an atomic action; the set of atomic actions; $N =  A $              |
| $\tau$                | action category: RELATIONAL or SEMANTIC                             |
| $g,  g $              | a group of actions fused into one FAO; its size                     |
| $\mathcal{V}^+$       | legal groups with at least one semantic action                      |
| $P, \mathcal{P}, P^*$ | a partition of $A$ ; all legal partitions; the one minimizing Eq. 1 |
| $f_g$                 | the function synthesized for group $g$                              |
| $b(g), b_s(g)$        | estimated execution tokens for $g$ : full scale, sample             |
| $D_s$                 | profiling sample drawn from the base tables                         |

per action rather than one function per query. The next two sections address execution and planning costs: the Optimizer further cuts execution costs by grouping actions (§4), and the Executor cuts planning costs by reusing cached functions (§5).

#### 4 OPTIMIZER: OPTIMIZING FAO PLANS

Table 1 summarizes the notation used in this section. The **KATHDB-FAO** Optimizer first turns a query sketch into an atomic plan by resolving each action’s input and output relations, linking the actions through them, and generating an FAO for each action. At this point, the FAO is only a function specification without an implementation. For example, the sketch in Figure 4 yields the atomic plan in Figure 5. Executing the atomic plan directly leads to two problems. First, it is expensive: the plan often does unnecessary work, such as running costly semantic operations on rows that are later dropped, and in our experiments an atomic plan can be up to 7× more expensive than an optimized one.

Second, an atomic plan can be inaccurate: FAO bodies are synthesized in isolation and in topological order, so a consumer may discover during code generation that its producer’s output does not contain what it needs. In Example 1, for example, the count node may expect a formal column holding Formal or Non-Formal, while the producer only emits a free-text style such as business casual. We address the first problem next in Q2 and the latter in Q3 where we describe contracts at the end of the section.

**Q2: How to apply logical rewrites to the atomic plan?** The same query admits many semantically equivalent plans (Figure 5). Plan A applies a VLM to the same image records only once, answering two questions in a single prompt (e.g., deciding whether an item is *formal* and whether it is *non-patterned* together) instead of one call each. Plan B fuses the two classification actions, the per-brand count, and the count  $\geq 5$  filter into one program that stops classifying a brand at its fifth qualifying item, further reducing VLM invocations.

**Grouping as the rewrite primitive.** To optimize the query plan, **KATHDB-FAO** needs to search over ways to combine individual actions together and over implementations of the resulting groups, which is an unbounded space. To efficiently find a good query plan, the key idea behind **KATHDB-FAO** is to decouple the questions of which actions to group together and how to implement the resulting logic. **KATHDB-FAO** optimizes a query by first only identifying good ways to group individual actions. Grouping offers no benefit by itself; its value is the optimization it *enables* later during code synthesis. Our design is for **KATHDB-FAO** to use grouping as the only rewrite primitive, while taking the atomic plan’s shape as given. This approach effectively limits our search space and avoids

the need to check for equivalence, since the grouping rewrite does not impact the logic of the plan. Logical reordering (e.g., operator or join reordering) could be considered as well, but we leave it to future work to keep the initial design simple. Experimental evaluation (§6.3) already shows the power of our approach.

More formally, given the atomic plan DAG over actions  $A = \{a_1, \dots, a_N\}$ , a grouping merges a subset  $g \subseteq A$  into a single FAO whose body realizes the combined logic of the members of  $g$ , implemented jointly as one function.

**Definition 4 (Legal Grouping).** A group  $g$  is *legal* iff its induced subgraph is *convex*: no non-member lies on a data-flow path between two members (equivalently, merging the members keeps the DAG acyclic). A *partition*  $P$  covers  $A$  with disjoint legal groups (a one-atom group is a *singleton*), and the set  $\mathcal{P}$  of legal partitions is finite but combinatorial in  $N$ .

**Cost model.** The second question is how to decide which grouping of actions yields a better plan. What should be the optimization objective? Here, the answer is simpler: In multimodal plans, model inference is known to dominate the monetary cost and the latency [17, 23]. The goal of **KATHDB-FAO** is thus to find a plan that minimizes inference costs and, more specifically, the overall token usage of the plan. At the same time, a simpler group is less likely to fail during synthesis and more likely to be reused, saving the cost of synthesizing the same optimization again (§6.5). Given these contradictory goals, our Optimizer minimizes the plan’s total execution token usage among legal partitions, subject to a size cap on each group:

$$P^* = \arg \min_{P \in \mathcal{P}} \sum_{g \in P} b(g) \quad \text{s.t.} \quad \forall g \in P: |g| \leq K, \quad (1)$$

where  $b(g)$  is the group’s estimated execution token usage and  $K$  limits the number of actions fused into one group. Group size  $|g|$  is a proxy for complexity: a larger group exposes more optimization opportunities but is more complex and less likely to be reused.  $K$  is an optimization parameter<sup>2</sup>.

**Search space.** To further reduce the search space, the Optimizer distinguishes between relational and semantic actions, as annotated in the original query sketch. Since only semantic actions cost tokens, grouping relational actions on their own cannot reduce execution cost. The Optimizer therefore searches only over  $\mathcal{V}^+$ : legal groups that contain at least one semantic action and satisfy  $|g| \leq K$ . Relational actions are never grouped among themselves. A relational action may be included in a group that also contains a semantic action, and whether it is included is decided by the plan search like any other grouping choice: in Plan B (Figure 5), the per-brand count and the final filter join the two classifiers because that enables the early exit, while the join and price filters stay singletons because fusing them would save no model calls. Every action not placed in a group remains an atomic FAO. The Optimizer then enumerates the legal partitions built from  $\mathcal{V}^+$ .

**Estimating query plan cost.** The hardest problem in the optimization process is how to estimate query plan costs. A semantic operator’s output, and even its selectivity, depend on unpredictable model behavior, so  $b(g)$  cannot be estimated reliably without generating code and grounding it in data [27, 39]. To obtain good

<sup>2</sup>We set  $K = 5$  empirically: larger  $K$  did not improve execution cost significantly, but increased plan-generation cost due to a larger search space.

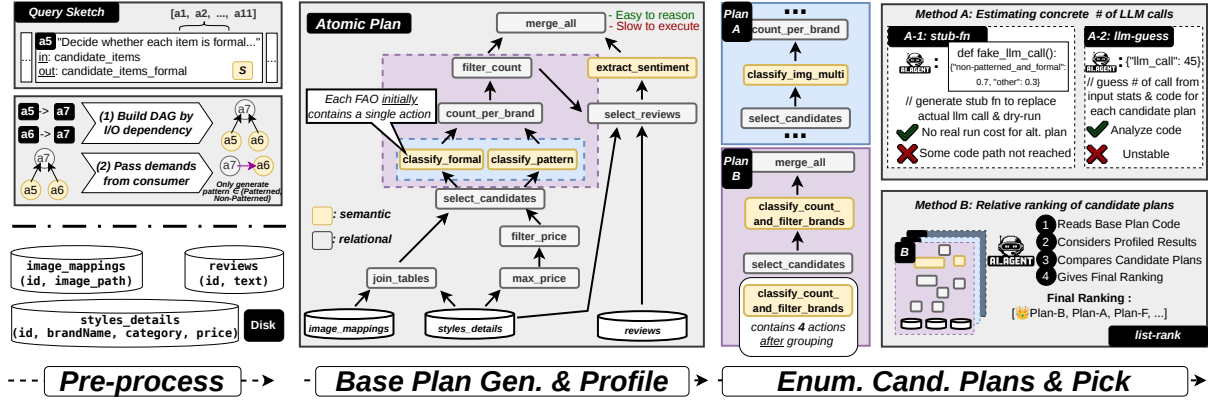


Figure 5: The Optimizer in KATHDB performs plan rewrites.

estimates, our approach is to have the Optimizer profile the initial, atomic plan once and use the resulting information during query optimization when comparing various groupings of this initial, atomic plan. Specifically, at the start of each query, the Optimizer draws a small uniform sample  $D_s$  of the base tables,<sup>3</sup> asks the Code Generator to generate code for every *atomic* operator, and runs it on  $D_s$  with real model calls. This materializes every intermediate relation at sample scale and records each operator’s estimated selectivity and its number of model calls on  $D_s$ . The intermediate relations also give the Code Generator real values and schemas as context. Estimating result sizes from a sample is known to be a hard problem [6, 16] and is harder here because semantic operators behave unpredictably; we assume  $D_s$  is representative and leave more accurate sampling to future work. We consider three strategies to estimate the cost of a plan, illustrated on Plan B in Figure 5. We later show in the evaluation section that the last strategy yields the best plans with the lowest planning overhead.

**Stub execution (stub-fn).** The Optimizer asks the Code Generator to synthesize  $f_g$ , the function implementing each new group  $g$ , replacing model inference calls with stubs. To generate the stubs, the Code Generator estimates the distribution over each model call’s outputs from the profiling sample; for the formal-non-patterned classifier this might be `{non-patterned_and_formal:0.7, other:0.3}`. The Code Generator inserts a stub helper encoding this distribution into the new FAO (e.g., `fake_llm_call()` in Figure 5), then runs  $f_g$  on the group’s input cached from profiling, with every model call replaced by the stub, which samples a result instead of calling the model. This dry-run issues no real model calls and yields  $b_s(g)$ , the cost on the sample, which the Optimizer scales linearly by the ratio of full-scale to sample-scale row counts of the stub’s input relation to obtain  $b(g)$ . The estimate is concrete but inherits the sampling problem: the control-flow paths that realize a saving may never execute at sample scale. An early exit that stops classifying a brand’s products once the brand already qualifies, for instance, rarely triggers when the sample holds only a few products per brand, so the stub counts a call for nearly every sampled row and the fused group looks no cheaper than running its actions separately.

**Analytic LLM estimation (llm-guess).** In this approach, the Optimizer does not run  $f_g$ : it still asks the Code Generator to produce code for each new group, and then asks an LLM to predict  $b(g)$  from the code, without executing it. The LLM sees the generated function, the profiling statistics (per-operator selectivities,  $D_s$  row counts, value distributions), and the full input sizes (a base table’s size is given; an intermediate relation’s is the base size times the profiled selectivities of the operators above it). For each call site it returns an estimated full-scale call count with a one-line rationale, which the Optimizer sums. By reasoning about early exits and skipped work in the code, it may identify optimization opportunities that the stub approach misses. Its weakness is non-determinism: estimates vary across calls (e.g.,  $\approx 30$  calls in one run and  $\approx 45$  in another for the same group), and summed across a plan this noise can alter which plan looks cheapest from run to run.

**Partition ranking (list-rank).** Selecting a partition needs only the *relative order* of candidates, and LLMs are more reliable at comparison than at estimation [44], so in this last approach, which we call list-rank, the Optimizer avoids per-group cost estimates altogether. The Optimizer shows the LLM the rendered plan (operator descriptions, the code generated for the atomic operators during profiling, profiled cardinalities, selectivities, and value statistics) and up to  $w$  candidate partitions, and asks which partition enables the lowest total execution token usage, breaking ties toward fusing *fewer* actions. With more than  $w$  candidates, the Optimizer runs a tournament that ranks chunks of  $w$  and recurses on the winners ( $w=2$  is pairwise comparison). In this approach, the LLM ranks Plan B above Plan A by identifying analytically the optimization opportunity B provides. Two knobs trade plan-generation cost against ranking quality (§6.3): the *ranking width*  $w$ , where a larger  $w$  compares more candidates per call and lowers cost but can degrade quality once the prompt crowds the context window, and the *code-generation timing*, where a fused group’s code can be generated during planning to help with ranking plans, deferred entirely to execution, or generated for planning and then regenerated during execution (once the entire output from input FAOs gets computed).

**Traversal strategies.** The two numeric estimators (stub-fn, llm-guess) produce additive per-group costs, so plan selection is a search over partitions of  $A$  minimizing  $\sum_{g \in P} b(g)$  over  $\mathcal{V}^+$ , with the size cap of Eq. 1 enforced during the search; the Optimizer uses a *DP*

<sup>3</sup>Empirically, at most 50 rows per base table.

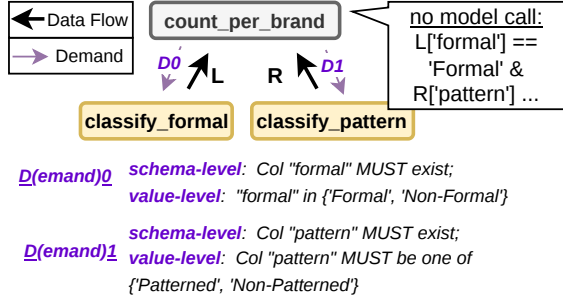


Figure 6: Top-down demand propagation in Example 1.

traversal for these. In the list-rank approach, the Optimizer obtains no per-group cost, so it selects among whole candidate partitions. *DP traversal.* In the DP traversal, the Optimizer finds the optimal partition under the estimates by dynamic programming over subsets of  $A$ , where  $dp[S]$  is the minimum total cost to partition  $S \subseteq A$ :

$$dp[S] = \min_{g \subseteq S, g \in V^+} (dp[S \setminus g] + b(g)), \quad dp[\emptyset] = 0. \quad (2)$$

It runs in  $O(3^N)$  time and, more importantly, must score every feasible group it touches, incurring high plan-generation cost.

*list-rank enumeration.* Because the list-rank approach never scores an individual group, the Optimizer enumerates all legal partitions built from  $V^+$  and ranks them against each other. The cap  $K$  on group size is what keeps this exhaustive enumeration practical: it bounds the number of legal groups, and thus the number of partitions, far below the unrestricted combinatorial count in  $N$ .

The output of query optimization is a new plan where some actions have been grouped. Each non-singleton group becomes a fused FAO that carries its member atomic actions with their descriptions, inputs, and outputs, together with the optimization rationale the Optimizer recorded for creating this group, which later steers code generation (or regeneration) during query execution.

**Q3: How to derive FAO contracts?** Because FAO bodies are synthesized independently, no fixed algebra defines the contract between adjacent operators, so a producer may emit values in a form its consumer cannot use, forcing the consumer to reconcile them with expensive LLM reasoning. Consider `count_per_brand` in Figure 6 and its two producers: `classify_formal` decides whether an item is formal and `classify_pattern` whether it is patterned. With unconstrained free-text outputs the two classifiers may describe a style in many ways (e.g., `business casual`, `smart`, `striped`), so the count node must interpret each item’s labels with an LLM, one call per item, and the count itself becomes a semantic action. The query fixes both domains in advance, so the count node demands that `classify_formal` emit {Formal, Non-Formal} and `classify_pattern` emit {Patterned, Non-Patterned}, and counts the qualifying items with a string test and no model call. Unlike the sentiments in §3, whose domain is open and known only after extraction has run, here the consumer can name the domain before any code runs, so the count node stays relational. The Optimizer derives these contracts by traversing the DAG from root to leaves, attaching to each node a set of *demands*. A demand maps an input relation to what the consumer expects from it at the *schema level* (the columns and types the consumer reads) and the *value level* (the set, range, or predicate those columns must satisfy). At

the root, the Optimizer asks the LLM to state the requirements on the final output based on the NL (e.g., an `id` column must exist), then propagates them downward: each node receives its description, its input relation names, and the demands accumulated from downstream consumers, and emits per-input demands pushed to the children producing those inputs. The Optimizer propagates top-down rather than bottom-up to capture downstream operator constraints enabling more optimizations.

## 5 EXECUTOR: GENERATING AND REUSING FUNCTIONS

The Executor interleaves function generation with execution. The Optimizer gives it a DAG of FAOs, in which singleton FAOs already carry an implementation from profiling, while grouped FAOs do not. The Executor walks the DAG in topological order. We assume the materialized data fits in memory and leave spilling to disk to future work. We observe two challenges in this process. First, generating plans is expensive: on the E-Com scenario, synthesizing every plan from scratch costs about 7.2× what execution costs at a small scale factor, since a cheap model is used for the operators while a more expensive one is used for the optimization (§6). Second, generation is not consistent: the Code Generator does not always write the same implementation for the same group when we repeat a query.

**Q4: How can we lower plan generation cost and help KATHDB-FAO reliably optimize similar queries?** We observe that the above two challenges are both related to function synthesis. Queries in a workload are often similar, but the Code Generator still writes new code with the same logic for each query. Each re-synthesis costs money, and the logic may change from run to run. Reusing one implementation addresses both challenges: it removes the synthesis call, and it keeps the logic stable across queries.

**FAO caching.** KATHDB-FAO caches an FAO’s implementation and reuses it in later queries. A newly generated FAO is tied to one query: its body hardcodes that query’s values, such as a threshold, a label, or a prompt string. Caching therefore works in two steps. During code generation, the Code Generator keeps these values inline, so it writes concrete working code, including the data-dependent control flow that yields the optimized execution. When an FAO is saved, the Executor then makes the function reusable by turning the hard-coded values into typed parameters with a short specification and a usage example. The logic of the function body stays the same. The Function Manager stores each function in its own directory with an implementation (`fn.py`) and a document (`fn.md`) that holds a summary and a usage example. Showing the entire documentation for each function at every step of query evaluation costs too many tokens, though. Following earlier work [4, 39], the Function Manager therefore reveals documents progressively: the Parser sees only function description summaries, and the Code Generator sees the full description (i.e., `fn.md`) but only for the functions relevant to the FAO being synthesized. Caching raises two new questions: what functions should the library contain and when should a cached function be used in planning?

**What to cache.** One option is to prefill the library with expert-written semantic and relational operators (e.g., `sem_filter`) [23]. We find that this does not help on our workloads (§6.5): the Code

Generator can write these simple operators in a few lines, so showing their documentation costs more tokens than reuse saves. Instead, our approach is to start from an empty library and fill it with functions that the system generates over time. Saving every generated FAO, however, is also wasteful: a plan contains many FAOs and only some of them are worth keeping, because most implement simple logic, such as a single `rel_select`, which, again, is as cheap to write as it is to reuse. To decide which ones to keep, the Code Generator asks the LLM for a save recommendation based on two criteria: the function (i) has non-trivial control flow and (ii) implements an algorithm that cuts token cost, such as an early exit. We call this policy *llm-judge*. Empirically, *llm-judge* is cheaper than saving every generated function (*save-all*), and it is the policy we adopt in **KATHDB-FAO**. Under *save-all*, the Executor needs to invoke an LLM to turn every function into a reusable one, including trivial functions such as a single `rel_select`, and the larger library it builds makes later queries spend more input tokens reading function documentation.

**When to reuse.** In the first approach, *reuse-after*, the Code Generator reuses functions after parsing. The Parser produces the usual atomic sketch (§3) and marks each action with potentially relevant functions. The Optimizer unions those functions whenever it groups actions into an FAO (§4). The Code Generator then has the option to reuse a matching cached function instead of writing new code for an FAO. In this approach, **KATHDB-FAO** saves only code generation costs: by the time function reuse happens, **KATHDB-FAO** has already built the atomic plan and paid the cost of searching over groupings. In this approach, the Parser may also struggle with identifying potential functions: a cached function often comes from a grouped node, so it covers more than one atomic action, and a Parser that reads one action at a time will not necessarily identify the function as a good match. In the second approach, *reuse-before*, the Function Manager shows the Parser relevant cached functions while it generates the sketch. In this approach, we also relax the constraint that the Parser generate atomic actions when finding matching cached functions. For example, the Parser may decide that a cached function fits an action best even though the function is coarser than a semantically atomic action, and may emit one larger action bound to that cached FAO. The Parser deviates from atomicity *only when* a cached function is a strong match, judged by the Parser LLM; otherwise it decomposes atomically as usual. With *reuse-before*, the base plan itself changes. The cached FAO replaces several atomic actions before optimization, so the Optimizer searches over fewer nodes. Reusing the implementation directly also removes the run-to-run variance of re-synthesis. The risk is that the Parser picks a function that does not fit and applies it in the wrong place. We evaluate both strategies in §6.5.

## 6 EVALUATION

In this section, we evaluate **KATHDB** with **KATHDB-FAO** over SemBench [15] and show that it outperforms a variety of baselines across scenarios in quality, planning cost, and execution cost. We ask five questions: (Q0) How does **KATHDB** compare against existing multimodal data systems in end-to-end quality and efficiency (Section 6.1)? (Q1) How does action granularity affect results (Section 6.2)? (Q2) How does grouping as a rewrite affect results

(Section 6.3)? (Q3) How does deriving contracts among nodes affect results (Section 6.4)? And (Q4) How does function reuse across queries affect results (Section 6.5)? **Dataset:** We evaluate **KATHDB** on SemBench [15], a benchmark for semantic data systems, whose operators invoke model inference over data that may be multimodal. Of its five scenarios, we use the three that span tabular, text, and image data: *Movie* [36] (sentiment analysis of Rotten Tomatoes reviews), *E-Com* [1] (fashion products as text and images), and *MMQA* [31] (cross-modal QA over tables, text, and images). Each scenario defines a scale factor (sf) that caps the number of rows used. We add a fourth scenario, *Additional*<sup>4</sup>, which reuses the E-Com and MMQA data but ask more complex queries. Its ten queries split evenly: five of them are from E-Com (sf1000) and the other five are from MMQA (sf400).

**Evaluation Metrics:** Following SemBench [15] we score each query with the metric appropriate to its task and map all scores onto a common [0, 1] scale where higher is better. Retrieval queries are scored by F1, clustering and grouping queries by the Adjusted Rand Index, aggregation queries by  $\max(0, 1 - \text{relative\_error})$ , and ranking queries by  $\max(0, \rho)$ , where  $\rho$  is Spearman’s rank correlation. We refer to this common per-query score as **Quality**, which lets us compare and aggregate across query types; we aggregate per scenario following SemBench.

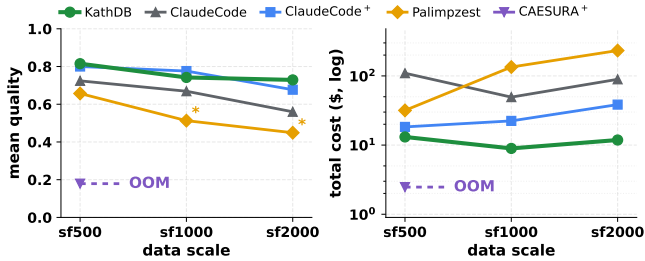
**Models:** For plan generation we use Claude-Opus-4.7 unless otherwise noted. The default AI-operator model is gpt-4o-mini unless otherwise specified.

**Baselines:** We compare **KATHDB** against five baselines: ① **Generative systems** take NL queries and generate the execution plan internally, so we measure both plan-generation and execution cost: (1) **CAESURA**<sup>+</sup> [33]: CAESURA translates NL queries into multi-step plans over modality-specific operators. Its original PLM-based operators frequently timed out, so we replaced them with LLM-based ones. (2) **ClaudeCode** [3]: Anthropic’s coding agent, which autonomously writes and executes Python to answer each query end-to-end. Out of the box, it can only invoke its own Claude models, whereas all other systems use the cheaper gpt-4o-mini as their AI operator. (3) **ClaudeCode**<sup>+</sup>: ClaudeCode with access to gpt-4o-mini, so the Python it writes can issue LLM calls to the same, cheaper model the other systems use. ② **Programmatic systems** require that an expert pre-write a SQL or Python plan rather than translating NL. We use the plans SemBench provides, so these systems incur no plan-generation cost, but this shifts the cost to a human expert that generative systems such as **KATHDB**, ClaudeCode<sup>+</sup>, and CAESURA<sup>+</sup> avoid: (4) **Palimpzest** [17]: a declarative multimodal analytics engine whose plan is a procedural Python program written against its DataFrame API. It is paired with Abacus [27], its query optimizer, with the “MaxQuality” policy set. Since its published SemBench numbers used different hardware and models, we re-run it in our environment under our standard model configuration. (5) **BigQuery** [9]: Google’s cloud data warehouse, whose SQL dialect includes AI functions that apply an LLM to each row. We report its numbers directly from SemBench [15]; note that these runs use Gemini-2.5-Flash [7], whose per-token price is higher than that of gpt-4o-mini (2× for input, ~4× for output). **Setup:** All experiments run on a single NVIDIA L40S GPU with 2 CPU cores

<sup>4</sup>The queries are available at <https://github.com/xertxiao/KathDB>.

**Table 2: Per-scenario results on SemBench (serial execution, except BigQuery). Each number is averaged over three runs per query, then over queries. Bold=best, underline=2nd, ranked among the generative systems only. \* =includes at least one query that timed out at the 12h cap, scored 0. OOM means a system fails due to out-of-memory in the current setup. BigQuery averages exclude the E-Com queries it cannot express (Q10, Q11).**

| System                  | Additional (sf1000, 400) |             |              |              |             | MMQA (excl. Q5, sf200) |             |              |              |             | E-Com (excl. Q14, sf500) |             |              |              |             | Movie (sf2000) |             |              |              |             |
|-------------------------|--------------------------|-------------|--------------|--------------|-------------|------------------------|-------------|--------------|--------------|-------------|--------------------------|-------------|--------------|--------------|-------------|----------------|-------------|--------------|--------------|-------------|
|                         | plan t↓                  | plan \$↓    | exec t↓      | exec \$↓     | qual.↑      | plan t↓                | plan \$↓    | exec t↓      | exec \$↓     | qual.↑      | plan t↓                  | plan \$↓    | exec t↓      | exec \$↓     | qual.↑      | plan t↓        | plan \$↓    | exec t↓      | exec \$↓     | qual.↑      |
| KATHDB-FAO              | <u>125.2</u>             | <b>0.73</b> | <b>119.4</b> | <b>0.030</b> | <b>0.92</b> | 53.6                   | <b>0.34</b> | <b>134.3</b> | <b>0.031</b> | <b>0.85</b> | 96.1                     | <b>0.57</b> | 399.1        | <b>0.079</b> | <u>0.77</u> | 63.3           | <b>0.38</b> | 217.6        | <u>0.004</u> | <b>0.81</b> |
| CAESURA <sup>+</sup>    |                          |             | OOM          |              |             | 69.4                   | <u>0.89</u> | <u>252.1</u> | 0.154        | 0.53        | <b>54.5</b>              | <b>0.83</b> | <b>187.9</b> | <u>0.174</u> | 0.20        | <b>28.7</b>    | <u>0.48</u> | <u>147.5</u> | <b>0.003</b> | 0.44        |
| ClaudeCode              | N/A                      | N/A         | <u>238.6</u> | 4.399        | 0.86        | N/A                    | N/A         | 343.0        | 34.676       | <u>0.70</u> | N/A                      | N/A         | <u>370.6</u> | 10.520       | 0.72        | N/A            | N/A         | <b>85.5</b>  | 1.402        | <u>0.81</u> |
| ClaudeCode <sup>+</sup> | <b>70.5</b>              | <u>1.33</u> | <u>713.2</u> | <u>0.201</u> | <u>0.92</u> | 82.3                   | 1.49        | 295.4        | <u>0.065</u> | 0.69        | <u>56.4</u>              | 1.19        | 851.1        | 0.176        | <b>0.79</b> | <u>33.8</u>    | 1.01        | 312.2        | 0.007        | 0.77        |
| Palimpzest              | not implemented          |             |              |              |             | N/A                    | N/A         | 5646.9*      | 7.161        | 0.78        | N/A                      | N/A         | 2848.8       | 2.402        | 0.66        | N/A            | N/A         | 13547.6*     | 1.309        | 0.52        |
| BigQuery (Gemini-2.5)   | not implemented          |             |              |              |             | N/A                    | N/A         | 30.3         | 0.142        | 0.32        | N/A                      | N/A         | 42.1         | 2.254        | 0.70        | N/A            | N/A         | 42.2         | 0.553        | 0.75        |



**Figure 7: E-Com scalability: mean quality (left) and total cost (right, log-scaled), at three scale factors. \* means at least one query timed out at the 12h cap.**

and 128 GB of memory. **KATHDB**, **CAESURA<sup>+</sup>**, and **ClaudeCode** all support human-AI interaction (e.g., clarification questions, interactive debugging); we **disable all human-in-the-loop channels** for a fair comparison.

## 6.1 (Q0) Main Result

**KATHDB-FAO yields the best results across scenarios.** Table 2 reports the per scenario results for all systems. Figure 1 plots this table (excluding **ClaudeCode<sup>+</sup>** for clarity): one point per system, with the average total cost per query on the x-axis (in USD, log scale) and the mean quality on the y-axis, both averaged unweighted over the three scenarios that all systems run (MMQA, E-Com, and Movie). Here, **KATHDB-FAO** uses its best configuration: atomic action granularity with grouping enabled. For **ClaudeCode**, planning is intertwined with execution, so we report all costs under execution and mark planning as N/A. **Palimpzest** and **BigQuery** execute pre-written expert plans and therefore incur no planning cost; because *Additional* is a new scenario we develop, neither has a plan for it, so both are marked *not implemented*. We exclude Q14 from E-Com and Q5 from MMQA because **Palimpzest** and **BigQuery** cannot express them, respectively [15]. **BigQuery** results are taken directly from SemBench [15], which uses Gemini-2.5-flash [7] rather than gpt-4o-mini. Overall, **KATHDB-FAO** achieves the best or tied-best quality on three of the four scenarios, and on E-Com trails **ClaudeCode<sup>+</sup>** by 0.02 while executing at less than half its cost.

**KATHDB-FAO scales best and with high quality:** Figure 7 shows quality (left) and total cost (right) for all methods except for **BigQuery** on the E-Com scenario (we select one representative scenario due to the high cost of this experiment on baseline systems). On

the left of Figure 7, quality drops for every system: we have more ground truth rows, so recall drops and overall quality falls with it; but **KATHDB-FAO** still holds the highest quality at the largest scale, and **Palimpzest** times out on some queries at both sf1000 and sf2000.<sup>5</sup> On the right, **KATHDB-FAO** is the cheapest system at every scale and its total cost does not grow much, while **Palimpzest** times out on its expensive pair-wise LLM join at both sf1000 and sf2000, which makes it expensive. **ClaudeCode<sup>+</sup>** spends steadily more as it handles more data because it inspects the result images itself, so its cost grows with the number of images it reads. **ClaudeCode** is cheaper for some queries as the agent sometimes decides to give up on queries that are too expensive, and thus sees a huge drop in mean quality. **CAESURA** runs out of memory at sf1000.

**KATHDB-FAO outperforms other generative systems in quality or cost.** **CAESURA<sup>+</sup>** frequently fails to produce correct plans: its quality drops to 0.53 on MMQA, 0.20 on E-Com, and 0.44 on Movie, versus **KATHDB-FAO**’s 0.85, 0.77, and 0.81. Off-the-shelf coding agents achieve good quality overall, but remain expensive. **ClaudeCode** gets competitive quality, but without cost-aware logical rewrites it processes records one at a time, leading to execution costs as high as \$34.68 on MMQA versus \$0.031 for **KATHDB-FAO**. Even when we give it direct access to gpt-4o-mini (**ClaudeCode<sup>+</sup>**), **KATHDB-FAO** matches or exceeds its quality on three of the four scenarios; on E-Com it trails by 0.02 (0.77 vs. 0.79) while executing at 2.2× lower cost. Across all four scenarios, **KATHDB-FAO** is cheaper than **ClaudeCode<sup>+</sup>** at both planning and execution, and up to 4.2× cheaper in total, because its logical rewrites, such as grouping, avoid excessive model calls during execution.

**KATHDB-FAO also outperforms programmatic systems in quality or cost.** **Palimpzest** runs a plan written by a human expert, the most expensive planning resource of all, and pays no planning cost at run time; yet it is among the costliest systems at execution (MMQA \$7.16, E-Com \$2.40) and times out on several queries, a few of which use extremely expensive pairwise LLM joins. **KATHDB-FAO** achieves higher quality on every shared scenario while being orders of magnitude cheaper by automatically realizing logical rewrites that the static expert plan does not. **BigQuery** also executes a fixed plan, but **KATHDB-FAO** achieves higher quality at lower execution cost on all three shared scenarios. On MMQA, for example, **KATHDB-FAO** achieves 0.85 quality at \$0.031 execution cost, compared with **BigQuery**’s 0.32 quality at \$0.142.

<sup>5</sup>For timed-out queries we include the cost they incurred with quality as 0.

Table 3: Impact of action granularity (*Additional*, *E-Com* and *MMQA* scenarios). Each number is averaged over three runs. Bold=best, underline=2nd within each group.

| Granularity   | Easy Queries |              | Hard Queries |              |
|---------------|--------------|--------------|--------------|--------------|
|               | exec \$↓     | qual.↑       | exec \$↓     | qual.↑       |
| atomic        | 0.080        | 0.873        | <b>0.099</b> | <b>0.453</b> |
| single-action | <b>0.078</b> | 0.846        | 0.272        | 0.433        |
| llm-decided   | 0.101        | <b>0.878</b> | <u>0.117</u> | 0.408        |

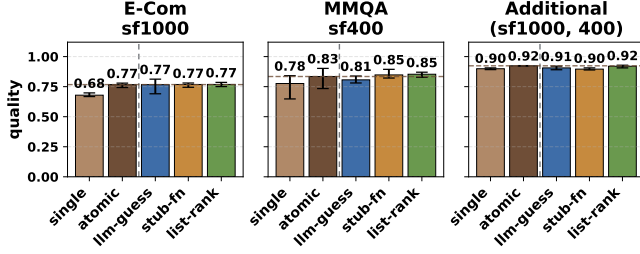


Figure 8: Quality for different plan cost comparison methods. Mean over three repetitions; whiskers show the range.

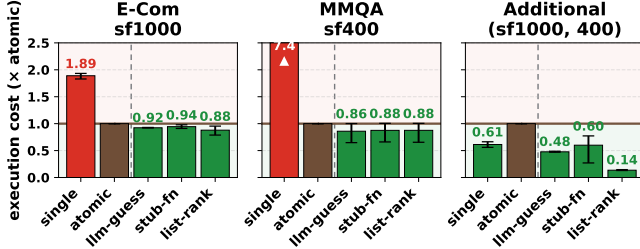


Figure 9: Execution cost, relative to the atomic plan, for the same configurations and scenarios as Figure 8. The horizontal reference line marks the atomic plan; bars beyond the y-axis range are marked ▲ with their rounded ratio.

## 6.2 (Q1) Action Granularity

Table 3 shows how action granularity affects end-to-end results over the *Additional*, *E-Com*, and *MMQA* scenarios, comparing three configurations of **KATHDB-FAO** that differ only in how the Parser decomposes queries: *single-action*, where the whole query becomes one action; *llm-decided*, where the model chooses where to split; and *atomic*, where we instruct the model to follow the semantic atomic rule to decompose the original query into actions (§3). We call a query *hard* if its quality averaged over the three granularities is below 0.6, and *easy* otherwise. Atomic achieves the highest quality on the hard queries, while all three configurations perform similarly on the easy queries. Single-action generates one or a few large-scoped functions in one shot, which can hurt quality because the system does not see intermediate query results (§3). Llm-decided, for some queries, encounters a similar problem as single-action by choosing a much coarser-grained decomposition. Thus, for the following experiments, we fix atomic as our granularity.

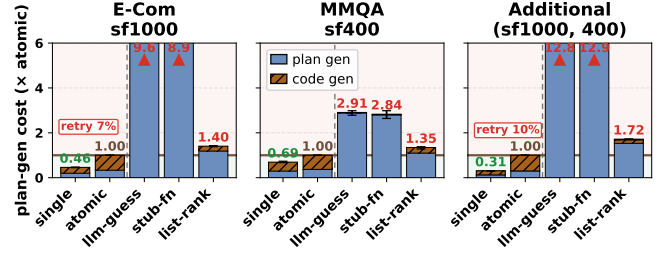


Figure 10: Plan-generation cost, relative to the atomic plan, for the same configurations as Figure 8. Bars beyond the y-axis range are marked ▲ with their rounded ratio.

## 6.3 (Q2) Grouping as a Rewrite

The next questions we ask are (1) whether grouping-as-rewrite (§4) reduces execution costs and (2) the best way to decide which grouping brings the most saving. We run **KATHDB-FAO** under five configurations. Two are baselines: *single*, which synthesizes the entire query as one FAO (§3) and *atomic*, which executes the atomic plan as-is, without any rewrites. Following §4, the remaining three configurations enumerate candidate grouped plans and evaluate them with *stub-fn*, *llm-guess*, or *list-rank*. For each configuration, we measure the aggregate plan quality and the aggregate execution and plan-generation costs, both reported relative to the atomic plan, on the *E-Com*, *MMQA*, and *Additional* scenarios, over three runs. Figures 8, 9, and 10 report quality, execution cost, and plan-generation cost for the five configurations; Figures 11 and 12 then fix the best-performing method (*list-rank*) and vary its configuration knobs, which we discuss at the end of this section.

**list-rank preserves quality while achieving the lowest execution cost.** Figure 8 shows that *list-rank* matches the atomic plan’s quality across all three scenarios. Figure 9 then shows that *list-rank* finds the cheapest plans overall, while Figure 10 shows that it does so with only a small additional planning cost. Recall from §4 that **KATHDB-FAO** enumerates candidate plans for each possible grouping (i.e., partitions of the base plan into a DAG of fused groups) and must select the one with the lowest estimated execution cost. Among the three evaluation methods, *llm-guess* is the weakest because it asks the LLM to predict, from a fused group’s code, the exact number of model calls the group will issue at full scale, and estimates the final plan cost as the sum of the per-group costs. Guessing such a concrete count per fused group is hard [44], and summing across groups compounds estimation errors, which can make expensive plans appear cheapest, so *llm-guess* often picks a less optimized plan. On *Additional*, the plans it picks execute at only 0.48× the atomic cost, whereas *list-rank*, the best method, reaches 0.14× (Figure 9, right). *stub-fn* runs each candidate plan on a small data sample to estimate the cost. The sample, however, is often too small to reach the control-flow paths that create the savings of a fused group; for example, an early-termination condition that is never triggered on a small data sample but is triggered frequently at full scale. As a result, *stub-fn* may underestimate the benefit of a fused group, and its plans reach only 0.60× the atomic cost on *Additional* (Figure 9, right). Both methods are also expensive: scoring every candidate group during the dynamic-programming search drives plan-generation cost to about 13× the atomic plan on

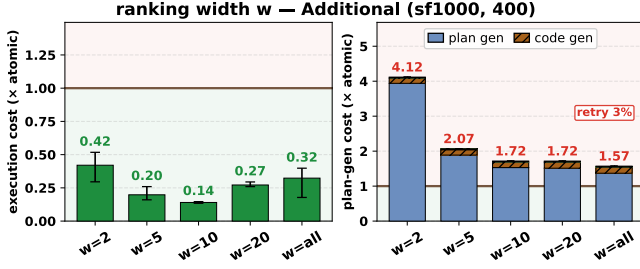


Figure 11: The ranking width  $w$  of list-rank: execution cost (left) and plan-generation cost (right), relative to the atomic plan (horizontal reference line). Quality varies by at most 0.04 across the knob settings and is skipped for space.

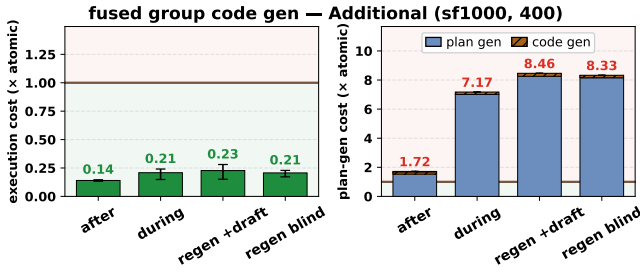


Figure 12: The fused-group code-generation timing of list-rank: execution cost (left) and plan-generation cost (right), relative to the atomic plan (horizontal reference line).

Additional (Figure 10, right). list-rank avoids per-group numbers altogether: it shows the LLM a set of candidate plans side by side and asks only which plan is cheapest, so the LLM produces a relative order rather than absolute estimates. This is both cheaper, keeping plan-generation cost within  $1.72\times$  of the atomic plan across the scenarios, and more accurate, since LLMs are better at ranking than at giving absolute numbers [44].

**Knobs for list-rank.** Having found list-rank to be the strongest method, we study its two configuration knobs. The first knob, the *ranking width*  $w$ , is how many candidate plans **KATHDB-FAO** shows the LLM in a single ranking call. Figure 11 shows that  $w=10$  gives the best trade-off: smaller widths require more ranking calls, while larger widths put too many plans into one prompt and can degrade the ranking [18, 40]. We therefore use  $w=10$  as our default. The second knob, *fused-group code-gen*, controls *when* fused-group code is written. Recall that list-rank ranks candidate partitions using their member atoms’ code and profiled statistics, before any fused-group code exists. We ask whether generating this code earlier helps the LLM choose a cheaper plan. Fixing  $w=10$ , we compare four settings (Figure 12): *after* generates fused-group code only for the winning plan at execution time; *during* generates code for every candidate during optimization and executes it as is; *regen+draft* regenerates that code at execution time using the earlier version as context; and *regen blind* regenerates it from scratch to avoid bias from the planning-time code. The latter three settings are much more expensive to plan because they generate code for *every fused group of every candidate*. Despite this extra cost, none achieves lower execution cost than *after*. Generating all fused groups also makes

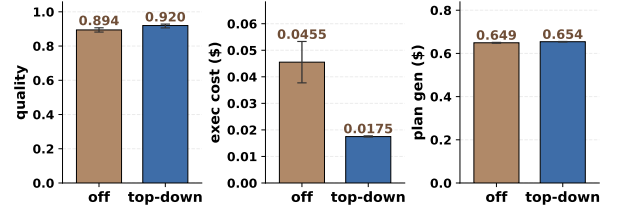


Figure 13: Effect of demands propagation evaluated over *Additional* scenario.

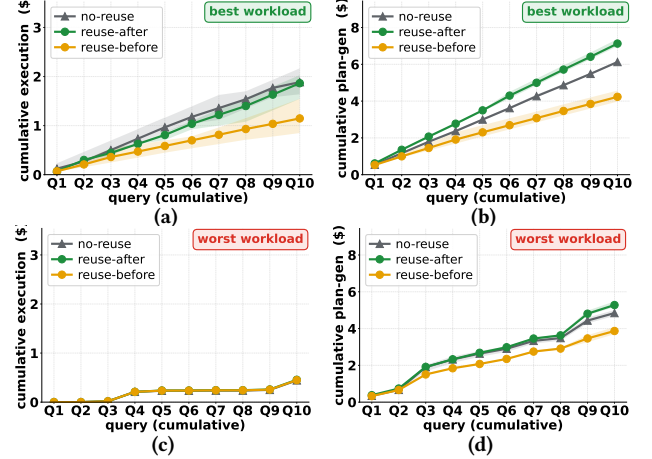


Figure 14: Effect of letting cached functions steer the Parser. Library policy fixed to *llm-judge*.

the ranking prompt much larger, exacerbating the long-context problem [18] and causing the optimizer to select less optimized plans. **KATHDB-FAO** therefore defaults to *after*, which is cheapest for both planning and execution.

#### 6.4 (Q3) Contracts among FAOs

Recall from §4 that, because each FAO’s code is synthesized independently, **KATHDB-FAO** generates a *contract* between adjacent FAOs: it propagates each consumer’s requirements (its *demands* on the schema and the values of its inputs) top-down through the DAG, so that a producer emits its output in the exact form its consumer can use (Figure 6). The next question we ask is whether this demand propagation helps. To study it in isolation, we hold everything else fixed: we disable grouping and run only the atomic plan. We then compare two settings: *off*, where no demands are propagated and each FAO is synthesized without any contract, and *top-down*, which propagates demands across the DAG. As shown in Figure 13, demand propagation improves quality and reduces execution cost. The reason is that demands tell each FAO exactly what its consumer needs, allowing generated code to avoid unnecessary semantic work. For example, when detecting whether a photo collection contains different objects, the producer can stop after observing a second distinct label instead of classifying every photo. Because demands are derived before grouping, the same contracts also apply between grouped FAOs. Plan-generation cost remains nearly unchanged because our prototype derives demands for all nodes in a single LLM call.

## 6.5 (Q4) Function Reuse

Finally, we ask whether FAO reuse helps across queries, and how reused functions should be incorporated into planning. Following §5, we start from an empty library and save functions under *llm-judge*. To study reuse, we fix the best configuration from §6.3 and run **KATHDB-FAO** on two workloads of ten queries each. In the *best* workload, all ten queries are very similar and differ in a single clause, representing the case where reuse has the most to gain. In the *worst* workload, the ten queries are intentionally drawn from E-Com, MMQA, and *Additional* to be as different from one another as possible. After each query we measure the *cumulative* plan-generation cost and the *cumulative* execution cost, both in \$. Quality is roughly stable across all settings on both workloads, so we omit it for space.

**Cached functions should steer planning.** The more important question is *when* a cached function is introduced into planning. Figure 14 compares two reuse strategies with the library policy fixed to *llm-judge*. In *reuse-after*, the Parser first produces the usual atomic plan and the Optimizer performs grouping as normal; only after the plan is fixed can the Code Generator reuse a cached function in place of generating new code. In *reuse-before*, the Parser is shown relevant cached functions up front and may temporarily relax semantic atomicity to emit a coarser-grained action that directly matches a cached FAO. This distinction matters because reuse-after can save only code generation: by the time reuse occurs, **KATHDB-FAO** has already constructed the atomic plan and paid the cost of exploring its grouping search space. reuse-before instead changes the plan itself. When the Parser matches a cached FAO, that FAO replaces several atomic actions before optimization, reducing the number of nodes and candidate groupings the Optimizer must consider. As a result, reuse-before consistently lowers plan-generation cost on both workloads, even relative to the no-reuse baseline. On the best workload, reuse-before also lowers execution cost (Figure 14a). Because successive queries are similar, an efficient implementation discovered for an earlier query can be reused directly for later ones, carrying its optimization across the workload. reuse-after cannot exploit this as effectively because the atomic decomposition and grouping decisions have already been made before the cached implementation is considered. On the worst workload, reuse-before still reduces planning cost, but provides little execution-cost benefit: the queries differ enough that previously generated functions rarely encode an optimization useful to the next query. Overall, the key benefit of reuse comes from exposing cached functions *before* optimization rather than merely using them as code-generation shortcuts afterward. We therefore use *llm-judge* with *reuse-before*: selectively retain useful implementations and let them steer parsing, which shrinks the subsequent search space and can also carry execution optimizations across similar queries.

## 7 RELATED WORK

**Semantic operators in data systems.** A first line of work extends the relational model with AI-powered operators: LOTUS [23], Palimpsest [17], DocETL [29], and ThalamusDB [11] expose operators such as *sem\_filter*, commercial systems provide equivalents [9, 30], and ELEET [34] adds multimodal joins and unions over text and tables, executed with a small pre-trained extraction

model rather than an LLM to keep execution time low. In all of these, the operator set is fixed by the system and the user composes a plan from it. **KATHDB-FAO** fixes no operator set: each plan node is a function synthesized during execution, so a plan can express logic that no predefined operator covers, and the user never needs an operator interface to ask a question. **Optimizing semantic operator plans.** A second line optimizes a plan once its operators are given. Abacus [27] searches over the physical implementations of a fixed logical plan of semantic operators; GALOIS [28] adapts relational rewrites such as predicate pushdown to SQL over LLM-generated tables, again within a fixed algebra; MOAR [39] optimizes DocETL [29] pipelines of map, reduce, and filter operators with over thirty rewrite directives, including model substitution, context truncation, operator fusion, and replacing an operator’s LLM call with synthesized code. **KATHDB-FAO** is complementary to Abacus: Abacus fixes the logical operators and searches over their implementations, whereas **KATHDB-FAO** searches over *which operators to merge*, so that cross-operator optimizations (e.g., Plan B’s early termination) become expressible at all; the model calls inside a synthesized function remain open to physical choices (a cheaper model of equal capability, majority voting [37]) that Abacus could search over. **KATHDB-FAO** differs from MOAR in three ways: it has no predefined operators (MOAR’s generated code is still an implementation of the same map), it processes multimodal data rather than only text documents, and it estimates a candidate plan’s cost without executing it on samples (KathDB-FAO only executes the atomic plan on a sample), which matters because plan generation is itself a dominant cost (§6).

**Generating code for query processing.** Generating code for a query rather than interpreting it is a long-standing idea. Compiled query engines [14, 20] turn a SQL plan into specialized low-level code, fusing adjacent operators into one loop to avoid interpretation overhead. CodexDB [32] renders each node of a SQL query tree as a natural-language step and asks a model to synthesize code for it, without optimizing the plan. CAESURA [33] and XMODE [21] translate an NL query into a multi-step plan over a fixed set of tools (e.g., vision-language models); Aryn [2] compiles an NL query into a script over a fixed document-operator set and rewrites it with fixed rules such as collapsing consecutive extractions, but does not search over alternative plans. VOCAL-UDF [43] synthesizes a UDF when a video query needs a concept its library lacks; Evaporate [5] synthesizes extraction code from a document sample instead of prompting per record, ensembling candidate functions by estimated quality. In all of these systems, what each node computes is fixed before any code is written. **KATHDB-FAO** instead searches over how to draw boundaries around functions, estimates the cost of each alternative, and picks one to execute.

## 8 CONCLUSION

We presented **KATHDB-FAO**, a query evaluation subsystem for multimodal DBMSs. **KATHDB-FAO** parses an NL query into fine-grained atomic actions, enabling careful reasoning over their actions and their inputs and outputs. It optimizes plans by grouping actions into bigger functions. It further reuses functions across queries to save plan generation cost. On SemBench, **KATHDB-FAO**

cuts execution cost by 58.8% on average across scenarios compared with the next best system, at comparable or better quality.

## **ACKNOWLEDGMENTS**

This work was supported in part by the NSF through award 2211133 and by Teradata.

## REFERENCES

- [1] Param Aggarwal. 2019. Fashion Product Images Dataset. <https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images-dataset>
- [2] Eric Anderson, Jonathan Fritz, Austin Lee, Bohou Li, Mark Lindblad, Henry Lindeman, Alex Meyer, Parth Parmar, Tanvi Ranade, Mehul A Shah, et al. 2024. The design of an llm-powered unstructured analytics system. *arXiv preprint arXiv:2409.00847* (2024).
- [3] Anthropic. 2025. Claude Code. <https://claude.com/product/claude-code> Accessed: 2026-03-16.
- [4] Anthropic. 2025. The Complete Guide to Building Skills for Claude. <https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf> Accessed: 2026-03-16.
- [5] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avani Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. 2023. Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes. *Proc. VLDB Endow.* 17, 2 (Oct. 2023), 92–105. <https://doi.org/10.14778/3626292.3626294>
- [6] Surajit Chaudhuri, Gautam Das, and Vivek Narasayya. 2007. Optimized stratified sampling for approximate query processing. *ACM Trans. Database Syst.* 32, 2 (June 2007), 9–es. <https://doi.org/10.1145/1242524.1242526>
- [7] Gheorghe Comanici, Eric Bieber, Mike Schaeckermann, et al. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. *arXiv preprint arXiv:2507.06261* (2025).
- [8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, Minneapolis, Minnesota, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
- [9] Google Cloud. 2025. BigQuery: Fully Managed Data Warehouse. <https://cloud.google.com/bigquery> Accessed: 2026-03-16.
- [10] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. 2017. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*. 2961–2969.
- [11] Saehan Jo and Immanuel Trummer. 2024. ThalamusDB: Approximate Query Processing on Multi-Modal Data. *Proc. ACM Manag. Data* 2, 3, Article 186 (May 2024), 26 pages. <https://doi.org/10.1145/3654989>
- [12] Daniel Kang, Peter Bailis, and Matei Zaharia. 2019. BlazeIt: optimizing declarative aggregation and limit queries for neural network-based video analytics. *Proc. VLDB Endow.* 13, 4 (Dec. 2019), 533–546. <https://doi.org/10.14778/3372716.3372725>
- [13] Daniel Kang, John Emmons, Firas Abuzaid, Peter Bailis, and Matei Zaharia. 2017. NoScope: optimizing neural network queries over video at scale. *Proc. VLDB Endow.* 10, 11 (Aug. 2017), 1586–1597. <https://doi.org/10.14778/3137628.3137664>
- [14] Konstantinos Krikellias, Stratis D Viglas, and Marcelo Cintra. 2010. Generating code for holistic query evaluation. In *2010 IEEE 26th International Conference on Data Engineering (ICDE 2010)*. IEEE, 613–624.
- [15] Jiale Lao, Andreas Zimmerer, Olga Ovcharenko, Tianji Cong, Matthew Russo, Gerardo Vitagliano, Michael Cochez, Fatma Özcan, Gautam Gupta, Thibaud Hottelier, et al. 2025. SemBench: A Benchmark for Semantic Query Processing Engines. *arXiv preprint arXiv:2511.01716* (2025).
- [16] Richard J. Lipton, Jeffrey F. Naughton, and Donovan A. Schneider. 1990. Practical selectivity estimation through adaptive sampling. In *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data* (Atlantic City, New Jersey, USA) (SIGMOD '90). Association for Computing Machinery, New York, NY, USA, 1–11. <https://doi.org/10.1145/93597.93611>
- [17] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, et al. 2025. Palimpsest: Optimizing ai-powered analytics with declarative query processing. In *Proceedings of the Conference on Innovative Database Research (CIDR)*. 2.
- [18] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. [https://doi.org/10.1162/tacl\\_a\\_00638](https://doi.org/10.1162/tacl_a_00638)
- [19] Oscar Moll, Manuel Favela, Samuel Madden, Vijay Gadepally, and Michael Cafarella. 2023. SeeSaw: Interactive Ad-hoc Search Over Image Databases. *Proc. ACM Manag. Data* 1, 4, Article 260 (Dec. 2023), 26 pages. <https://doi.org/10.1145/3626754>
- [20] Thomas Neumann. 2011. Efficiently compiling efficient query plans for modern hardware. *Proc. VLDB Endow.* 4, 9 (June 2011), 539–550. <https://doi.org/10.14778/2002938.2002940>
- [21] Farhad Nooralahzadeh, Yi Zhang, Jonathan Furst, and Kurt Stockinger. 2024. Multi-Modal Data Exploration via Language Agents. *arXiv preprint arXiv:2412.18428* (2024).
- [22] OpenAI. 2025. Introducing deep research. <https://openai.com/index/introducing-deep-research/>. Accessed: 2026-08-31.
- [23] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proc. VLDB Endow.* 18, 11 (July 2025), 4171–4184. <https://doi.org/10.14778/3749646.3749685>
- [24] Perplexity. 2025. Introducing Perplexity Deep Research. <https://www.perplexity.ai/hub/blog/introducing-perplexity-deep-research>. Accessed: 2026-08-31.
- [25] Mark Raasveldt and Hannes Mühleisen. 2019. Duckdb: an embeddable analytical database. In *Proceedings of the 2019 international conference on management of data*. 1981–1984.
- [26] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Marina Meila and Tong Zhang (Eds.), Vol. 139. PMLR, 8748–8763. <https://proceedings.mlr.press/v139/radford21a.html>
- [27] Matthew Russo, Chunwei Liu, Sivaprasad Sudhir, Gerardo Vitagliano, Michael Cafarella, Tim Kraska, and Samuel Madden. 2026. Abacus: A Cost-Based Optimizer for Semantic Operator Systems. *Proceedings of the VLDB Endowment* 19, 5 (Jan. 2026), 1060–1073. <https://doi.org/10.14778/3796195.3796215>
- [28] Dario Satriani, Enzo Veltri, Donatello Santoro, Sara Rosato, Simone Varriale, and Paolo Papotti. 2025. Logical and Physical Optimizations for SQL Query Execution over Large Language Models. 3, 3, Article 181 (June 2025), 28 pages. <https://doi.org/10.1145/3725411>
- [29] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G. Parameswaran, and Eugene Wu. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *Proc. VLDB Endow.* 18, 9 (May 2025), 3035–3048. <https://doi.org/10.14778/3746405.3746426>
- [30] Snowflake. 2025. Snowflake Cortex AISQL (including LLM functions). <https://docs.snowflake.com/en/user-guide/snowflake-cortex/aisql>, accessed 2026-07-12.
- [31] Alon Talmor, Ori Yoran, Amnon Catav, Dan Lahav, Yizhong Wang, Akari Asai, Gabriel Ilharco, Hannaneh Hajishirzi, and Jonathan Berant. 2021. Multi-Modal{QA}: complex question answering over text, tables and images. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=ee6W5UgQLa>
- [32] Immanuel Trummer. 2022. CodexDB: Synthesizing code for query processing from natural language instructions using GPT-3 Codex. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2921–2928.
- [33] Matthias Urban and Carsten Binnig. 2024. CAESURA: Language Models as Multi-Modal Query Planners. In *14th Conference on Innovative Data Systems Research, CIDR 2024, Chaminade, CA, USA, January 14-17, 2024*. www.cidrdb.org. <https://www.cidrdb.org/cidr2024/papers/p14-urban.pdf>
- [34] Matthias Urban and Carsten Binnig. 2024. Eleet: Efficient learned query execution over text and tables. *Proceedings of the VLDB Endowment* 17, 13 (2024), 4867–4880.
- [35] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [36] Andrea Villa. 2023. Clapper: Massive Rotten Tomatoes Movies & Reviews. <https://www.kaggle.com/datasets/andrezaa/clapper-massive-rotten-tomatoes-movies-and-reviews>
- [37] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=IPL1NIMMrw>
- [38] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [39] Lindsey Linxi Wei, Shreya Shankar, Sepanta Zeighami, Yeounoh Chung, Fatma Özcan, and Aditya G. Parameswaran. 2026. Multi-Objective Agentic Rewrites for Unstructured Data Processing. *arXiv:2512.02289 [cs.DB]* <https://arxiv.org/abs/2512.02289>
- [40] Guorui Xiao, Dong He, Jin Wang, and Magdalena Balazinska. 2025. Cents: A Flexible and Cost-Effective Framework for LLM-Based Table Understanding. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4574–4587.
- [41] Guorui Xiao, Enhao Zhang, Nicole Sullivan, Will Hansen, and Magdalena Balazinska. 2026. KathDB: Explainable Multimodal Database Management System with Human-AI Collaboration. In *Proceedings of the Conference on Innovative Data Systems Research (CIDR)*.
- [42] Enhao Zhang, Maureen Daum, Dong He, Brandon Haynes, Ranjay Krishna, and Magdalena Balazinska. 2023. EQUI-VOCAL: Synthesizing Queries for Compositional Video Events from Limited User Interactions. *Proc. VLDB Endow.* 16, 11 (July 2023), 2714–2727. <https://doi.org/10.14778/3611479.3611482>
- [43] Enhao Zhang, Nicole Sullivan, Brandon Haynes, Ranjay Krishna, and Magdalena Balazinska. 2025. Self-Enhancing Video Data Management System for Compositional Events with Large Language Models. *Proc. ACM Manag. Data* 3, 3, Article 215 (June 2025), 29 pages. <https://doi.org/10.1145/3725352>

- [44] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and Chatbot Arena. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (*NIPS '23*). Curran Associates Inc., Red Hook, NY, USA, Article 2020, 29 pages.